Date:
Exp No:7

Aim:Implementation banker's Algorithm.

Theory

Banker's Algorithm is a deadlock avoidance algorithm.
The algorithm checks whether allocating requested resources will leave the system in a safe state.

Deadlock

A deadlock occurs when processes are waiting indefinitely for resources held by each other.

Safe State

A system is in a safe state if there exists at least one **safe sequence** in which all processes can complete execution without causing deadlock.

Safe Sequence

A sequence of processes such that each process can obtain its required resources and complete execution.

Data Structures Used

1. **Available[m]**
   Number of available instances of each resource type.
2. **Max[n][m]**
   Maximum demand of each process.
3. **Allocation[n][m]**
   Resources currently allocated to each process.
4. **Need[n][m]**
   Remaining resource requirement of each process.

   $$Need = Max - Allocation$$

1.Write the c program OS_exp7.c

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resource types: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m];
   int avail[m], finish[n], safeSeq[n];
```

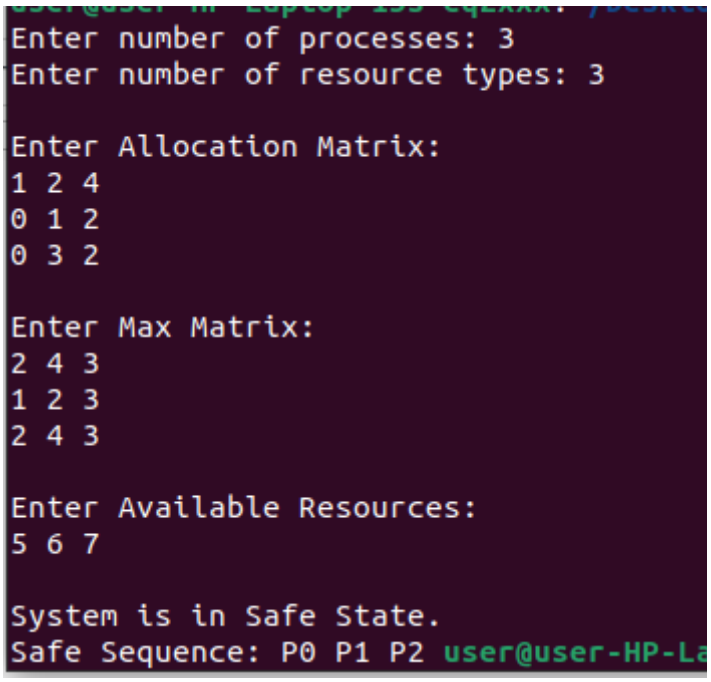
```
printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < n; i++)
   for(j = 0; j < m; j++)
      scanf("%d", &alloc[i][j]);

printf("\nEnter Max Matrix:\n");
for(i = 0; i < n; i++)
   for(j = 0; j < m; j++)
      scanf("%d", &max[i][j]);

printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++)
   scanf("%d", &avail[i]);

// Calculate Need Matrix
for(i = 0; i < n; i++)
   for(j = 0; j < m; j++)
      need[i][j] = max[i][j] - alloc[i][j];

for(i = 0; i < n; i++)
   finish[i] = 0;

int count = 0;

while(count < n)
{
   int found = 0;
   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         int flag = 1;
         for(j = 0; j < m; j++)
         {
            if(need[i][j] > avail[j])
            {
               flag = 0;
               break;
            }
         }

         if(flag)
         {
            for(k = 0; k < m; k++)
               avail[k] += alloc[i][k];

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
   }
```

```
        if(found == 0)
        {
            printf("\nSystem is in Unsafe State!");
            return 0;
        }
    }

    printf("\nSystem is in Safe State.\nSafe Sequence: ");
    for(i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);

    return 0;
}
```

2.Compile and Run

```
gcc OS_exp4_1.c
./a.out
```

3.Expected output



Result

The Banker’s Algorithm was implemented successfully.